

University
Politehnica
of Bucharest





Microsoft SQL Server (II)

Ș.L. Dr. Ing. Ciprian-Octavian Truică
ciprian.truica@upb.ro



Overview

- SQL and Transact-SQL Language

Transactions

- A transaction is a single unit of work
- If a transaction is successful, all the data modifications made during the transaction are
  - Committed
  - Become a permanent part of the database
- If a transaction encounters errors, all the data modifications made during the transaction are
  - Erased
  - Must be cancelled or rolled back



Transactions

- Autocommit transactions
  - Are the default transaction management mode of the SQL Server Database Engine
  - In this case, each individual statement is a transaction
  - Every Transact-SQL statement is committed or rolled back when it completes
    - A transaction is committed if a statement completes successfully
    - A transaction is it is rolled back if it encounters any error

Transactions

- Explicit transactions
  - Are explicitly started with the BEGIN TRANSACTION statement
  - Are explicitly ended with a COMMIT TRANSACTION or ROLLBACK TRANSACTION statement
- Implicit transactions
  - A new transaction is implicitly started when the prior transaction completes
  - Are explicitly completed with a COMMIT or ROLLBACK statement



Transactions

```sql
BEGIN TRANSACTION
SELECT region_name AS [Before Update] FROM regions WHERE region_id = 3;
UPDATE regions SET region_name = 'Oceania' WHERE region_id = 3;
SELECT region_name AS [After Update] FROM regions WHERE region_id = 3;
COMMIT TRANSACTION
SELECT region_name AS [After Commit] FROM regions WHERE region_id = 3;
GO

BEGIN TRANSACTION
SELECT region_name AS [Before Update] FROM regions WHERE region_id = 3;
UPDATE regions SET region_name = 'Oceania' WHERE region_id = 3;
SELECT region_name AS [After Update] FROM regions WHERE region_id = 3;
ROLLBACK TRANSACTION
SELECT region_name AS [After Rollback] FROM regions WHERE region_id = 3;
GO
```



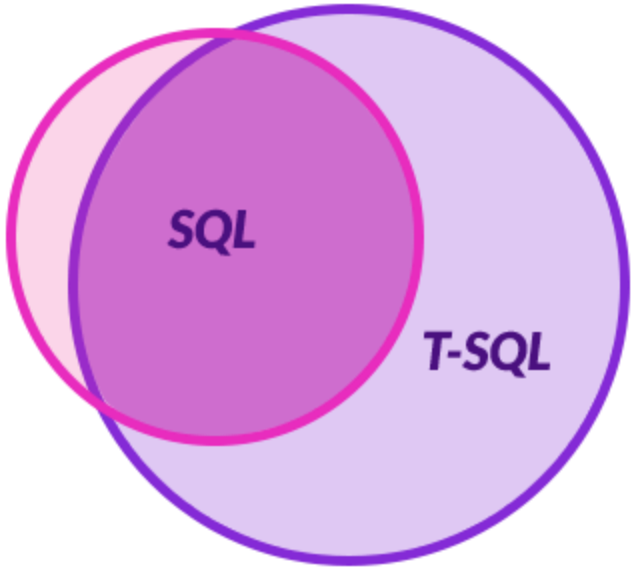
Transactions

- Batch-scoped transactions
  - Are applicable only to Multiple Active Result Sets (MARS)
    - MARS is a feature that allows the execution of multiple batches on a single connection
  - Are explicit or implicit transactions that start under a MARS session
  - If they are not committed or rolled back when a batch completes then they are automatically rolled back by SQL Server

Transact-SQL (T-SQL)

- Transact-SQL is a Microsoft SQL Server extension of SQL

- Transact-SQL contains:
  - Basic SQL clauses
  - Procedural programming abilities, (e.g., functions, procedures, triggers, exception handling, etc.)
  - Local variables



Data Types

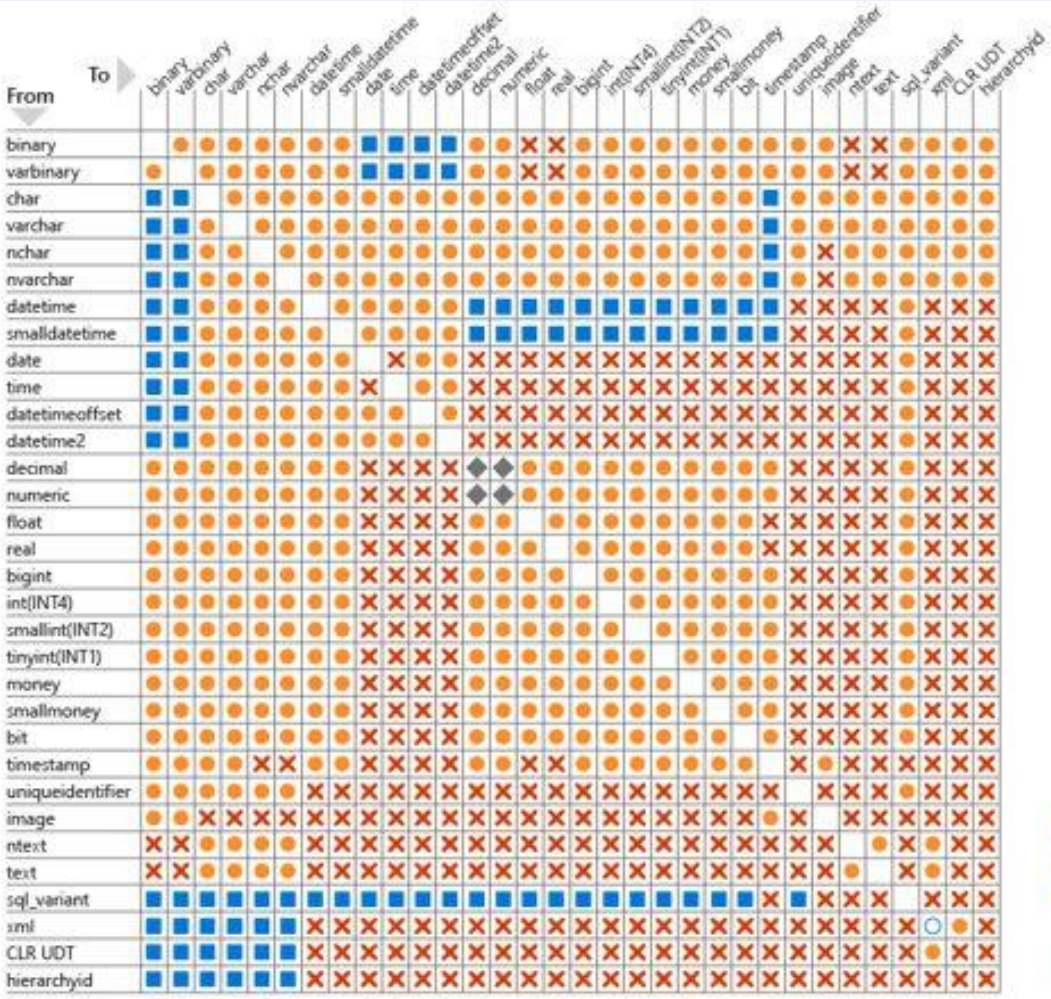
- SQL Server supplies multiple system data types
  - Exact numeric: SMALLINT, INT, BIGINT, NUMERIC(p,s), and DECIMAL(p,s), etc.
  - Approximate numeric: FLOAT(p), REAL
  - Character strings: CHAR(p), VARCHAR(p), TEXT
  - Unicode character strings NCHAR(p), NVARCHAR(p), NTEXT
  - Binary strings: BINARY(p), VARBINARY(p), IMAGE
  - Date and time: TIME, DATE, DATETIME2



Data Types

- User defined types
  - Can be created using either T-SQL or the Microsoft .NET Framework
  - Obtain their characteristics from the class' methods and operators
    - The class is created using one of the programming languages support by the .NET Framework

Data Types


From
To
binary
varbinary
char
varchar
nchar
nvarchar
datetime
smalldatetime
date
time
datetimeoffset
datetime2
decimal
numeric
float
real
bigint
int(INT4)
smallint(INT2)
tinyint(INT1)
money
smallmoney
bit
timestamp
uniqueidentifier
image
ntext
text
sql_variant
xml
CLR UDT
hierarchyid
Explicit conversion
Implicit conversion
Conversion not allowed
Requires explicit CAST to prevent the loss of precision or scale that might occur in an implicit conversion.
Implicit conversions between xml data types are supported only if the source or target is untyped xml. Otherwise, the conversion must be explicit.



Variables

- Variables can be either:
  - Standard
  - Table (temporal)

- Syntax: DECLARE @var_name data_type [= initial_value]

- Use SET/SELECT to assign a value

```
DECLARE @var int = 4
SELECT region_name FROM regions WHERE region_id=@var
GO

DECLARE @var int
SET @var = 4
SELECT region_name FROM regions WHERE region_id=@var
GO

DECLARE @var int
SELECT @var = region_id FROM countries WHERE country_id='ZW'
SELECT region_name FROM regions WHERE region_id=@var
GO
```



Data Definition Language

- Data Definition Language (DDL)
  - CREATE
  - ALTER
  - DROP

Data Definition Language

CREATE +

AGGREGATE
APPLICATION ROLE
ASSEMBLY
ASYMMETRIC KEY
AVAILABILITY GROUP
BROKER PRIORITY
CERTIFICATE
COLUMNSTORE INDEX
COLUMN ENCRYPTION KEY
COLUMN MASTER KEY
CONTRACT
CREDENTIAL
CRYPTOGRAPHIC PROVIDER
DATABASE
DATABASE AUDIT SPECIFICATION
DATABASE ENCRYPTION KEY
DATABASE SCOPED CREDENTIAL
DEFAULT
ENDPOINT
EVENT NOTIFICATION
EVENT SESSION
EXTERNAL DATA SOURCE
EXTERNAL LIBRARY
EXTERNAL FILE FORMAT
EXTERNAL RESOURCE POOL
EXTERNAL TABLE

FULLTEXT CATALOG
FULLTEXT INDEX
FULLTEXT STOPLIST
FUNCTION
INDEX
LOGIN
MASTER KEY
MESSAGE TYPE
PARTITION FUNCTION
PARTITION SCHEME
PROCEDURE
QUEUE
REMOTE SERVICE BINDING
RESOURCE POOL
ROLE
ROUTE
RULE

SCHEMA
SEARCH PROPERTY LIST
SECURITY POLICY
SELECTIVE XML INDEX
SEQUENCE
SERVER AUDIT
SERVER AUDIT SPECIFICATION
SERVER ROLE
SERVICE
SPATIAL INDEX
STATISTICS
SYMMETRIC KEY
SYNONYM
TABLE
TABLE (SQL Graph)
TABLE IDENTITY (Property)
TRIGGER
TYPE
USER
VIEW
WORKLOAD GROUP
XML INDEX
XML INDEX (Selective XML Indexes)
XML SCHEMA COLLECTION

+ OPTIONS

Data Definition Language

ALTER +

AGGREGATE
APPLICATION ROLE
ASSEMBLY
ASYMMETRIC KEY
AVAILABILITY GROUP
BROKER PRIORITY
CERTIFICATE
COLUMNSTORE INDEX
COLUMN ENCRYPTION KEY
COLUMN MASTER KEY
CONTRACT
CREDENTIAL
CRYPTOGRAPHIC PROVIDER
DATABASE
DATABASE AUDIT SPECIFICATION
DATABASE ENCRYPTION KEY
DATABASE SCOPED CREDENTIAL
DEFAULT
ENDPOINT
EVENT NOTIFICATION
EVENT SESSION
EXTERNAL DATA SOURCE
EXTERNAL LIBRARY
EXTERNAL FILE FORMAT
EXTERNAL RESOURCE POOL
EXTERNAL TABLE

FULLTEXT CATALOG
FULLTEXT INDEX
FULLTEXT STOPLIST
FUNCTION
INDEX
LOGIN
MASTER KEY
MESSAGE TYPE
PARTITION FUNCTION
PARTITION SCHEME
PROCEDURE
QUEUE
REMOTE SERVICE BINDING
RESOURCE POOL
ROLE
ROUTE
RULE

SCHEMA
SEARCH PROPERTY LIST
SECURITY POLICY
SELECTIVE XML INDEX
SEQUENCE
SERVER AUDIT
SERVER AUDIT SPECIFICATION
SERVER ROLE
SERVICE
SPATIAL INDEX
STATISTICS
SYMMETRIC KEY
SYNONYM
TABLE
TABLE (SQL Graph)
TABLE IDENTITY (Property)
TRIGGER
TYPE
USER
VIEW
WORKLOAD GROUP
XML INDEX
XML INDEX (Selective XML Indexes)
XML SCHEMA COLLECTION

+ OPTIONS

Data Definition Language

DROP [IF EXISTS] +

- AGGREGATE
- APPLICATION ROLE
- ASSEMBLY
- ASYMMETRIC KEY
- AVAILABILITY GROUP
- BROKER PRIORITY
- CERTIFICATE
- COLUMNSTORE INDEX
- COLUMN ENCRYPTION KEY
- COLUMN MASTER KEY
- CONTRACT
- CREDENTIAL
- CRYPTOGRAPHIC PROVIDER
- DATABASE
- DATABASE AUDIT SPECIFICATION
- DATABASE ENCRYPTION KEY
- DATABASE SCOPED CREDENTIAL
- DEFAULT
- ENDPOINT
- EVENT NOTIFICATION
- EVENT SESSION
- EXTERNAL DATA SOURCE
- EXTERNAL LIBRARY
- EXTERNAL FILE FORMAT
- EXTERNAL RESOURCE POOL
- EXTERNAL TABLE
- FULLTEXT CATALOG
- FULLTEXT INDEX
- FULLTEXT STOPLIST
- FUNCTION
- INDEX
- LOGIN
- MASTER KEY
- MESSAGE TYPE
- PARTITION FUNCTION
- PARTITION SCHEME
- PROCEDURE
- QUEUE
- REMOTE SERVICE BINDING
- RESOURCE POOL
- ROLE
- ROUTE
- RULE
- SCHEMA
- SEARCH PROPERTY LIST
- SECURITY POLICY
- SELECTIVE XML INDEX
- SEQUENCE
- SERVER AUDIT
- SERVER AUDIT SPECIFICATION
- SERVER ROLE
- SERVICE
- SPATIAL INDEX
- STATISTICS
- SYMMETRIC KEY
- SYNONYM
- TABLE
- TABLE (SQL Graph)
- TABLE IDENTITY (Property)
- TRIGGER
- TYPE
- USER
- VIEW
- WORKLOAD GROUP
- XML INDEX
- XML INDEX (Selective XML Indexes)
- XML SCHEMA COLLECTION

+ Object Name

Data Definition Language

- CREATE OR ALTER
- Command introduced since SQL Server 2016 for Functions, Stored Procedures, Triggers and Views
- In case the object exists, it will be modified and if not, it will be created



Data Control Language

- Data Control Language DCL:
  - GRANT
  - DENY
  - REVOKE

- DDL commands are used to grant, deny or revoke access to resources to principals

- Principals are **entities that can request SQL Server resources**



Data Control Language

- Grants permissions on an object to a principal
- The general concept is to GRANT <some permission> ON <some object> TO <some user, login, or group/role>

```
GRANT { ALL [ PRIVILEGES ] }
      | permission [ ( column [ ,...n ] ) ] [ ,...n ]
      [ ON [ class :: ] securable ] TO principal [ ,...n ]
      [ WITH GRANT OPTION ] [ AS principal ]
```

Data Control Language

- Denies a permission to a principal
- Prevents that principal from inheriting the permission through its group or role memberships
- DENY takes precedence over all permissions,
- Exception: DENY does not apply to object owners or members of the sysadmin fixed server role.

```
DENY   { ALL [ PRIVILEGES ] }
     | <permission>  [ ( column [ ,...n ] ) ] [ ,...n ]
    [ ON [ <class> :: ] securable ]
    TO principal [ ,...n ]
    [ CASCADE] [ AS principal ]
[;]
```

Data Control Language

- Removes a previously granted or denied permission.

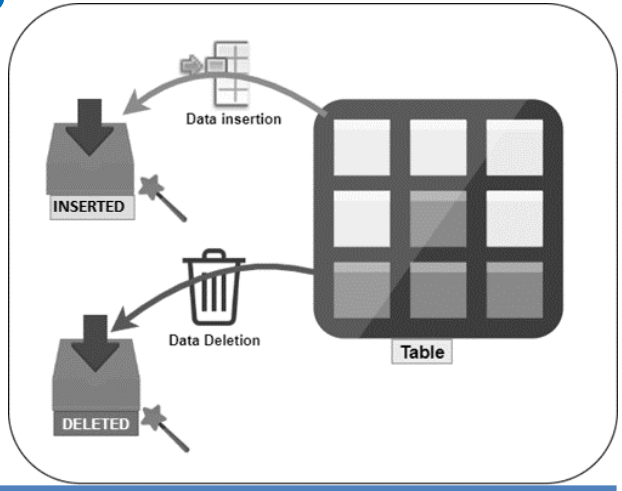
```
REVOKE [ GRANT OPTION FOR ]
      {
        [ ALL [ PRIVILEGES ] ]
        |
                permission [ ( column [ ,...n ] ) ] [ ,...n ]
      }
      [ ON [ class :: ] securable ]
      { TO | FROM } principal [ ,...n ]
      [ CASCADE] [ AS principal ]
```

Data Manipulation Language

- Data Manipulation Language (DML)
  - INSERT
  - BULK INSERT
  - UPDATE
  - DELETE
  - SELECT (considered DQL – Data Query Language)

Data Manipulation Language

- INSERT, UPDATE, DELETE, and DML TRIGGERS have access to 2 Virtual Tables: INSERTED and DELETED
- INSERTED: stores newly inserted rows
- DELETED: stores recently deleted rows



	INSERTED	DELETED
INSERT	New Values	NULL
UPDATE	Values after UPDATE	Value before UPDATE
DELETE	NULL	Deleted Values
DML TRIGER	New Values (Depended on DML on which it was triggered)	Old Values(Depended on DML on which it was triggered)

Data Manipulation Language
INSERT

- INSERT adds one or more rows to a table or a view in SQL Server

```
-- Syntax for SQL Server and Azure SQL Database

[ WITH <common_table_expression> [ ,...n ] ]
INSERT
{
        [ TOP ( expression ) [ PERCENT ] ]
        [ INTO ]
        { <object> | rowset_function_limited
          [ WITH ( <Table_Hint_Limited> [ ...n ] ) ]
        }
    {
        [ ( column_list ) ]
        [ <OUTPUT Clause> ]
        { VALUES ( { DEFAULT | NULL | expression } [ ,...n ] ) [ ,...n     ]
        | derived_table
        | execute_statement
        | <dml_table_source>
        | DEFAULT VALUES
        }
    }
}
[;]

<object> ::=
{
    [ server_name . database_name . schema_name .
      | database_name .[ schema_name ] .
      | schema_name .
    ]
  table_or_view_name
}

<dml_table_source> ::=
    SELECT <select_list>
    FROM ( <dml_statement_with_output_clause> )
      [AS] table_alias [ ( column_alias [ ,...n ] ) ]
    [ WHERE <search_condition> ]
        [ OPTION ( <query_hint> [ ,...n ] ) ]
```



Data Manipulation Language
INSERT

```sql
CREATE TABLE countries_copy(
    country_id   char(2),
    country_name varchar(30),
    region_id    int
);

INSERT INTO countries_copy VALUES (N'RO', N'Romania', 1);

INSERT INTO countries_copy VALUES
(N'MD', N'Moldova', 1),
(N'BG', N'Bulgaria', 1);

INSERT INTO countries_copy
SELECT country_id, country_name, region_id
FROM countries
WHERE region_id = 1;

WITH CountriesReg2(cid, cname, rid)
AS (
    SELECT country_id, country_name, region_id
    FROM countries
    WHERE region_id = 2
)
INSERT INTO countries_copy
OUTPUT inserted.country_id, inserted.country_name, inserted.region_id
SELECT cid, cname, rid
FROM CountriesReg2
ORDER BY cid desc;
```



Data Manipulation Language
INSERT

```sql
-- insert top 3 at random
WITH CountriesReg2(cid, cname, rid)
AS (
    SELECT country_id, country_name, region_id
    FROM countries
    WHERE region_id = 2
)
INSERT TOP(3) INTO countries_copy
OUTPUT inserted.country_id, inserted.country_name, inserted.region_id
SELECT cid, cname, rid
FROM CountriesReg2
ORDER BY cid DESC;

-- insert top 3 at ordered by ORDER BY
WITH CountriesReg2(cid, cname, rid)
AS (
    SELECT country_id, country_name, region_id
    FROM countries
    WHERE region_id = 2
)
INSERT INTO countries_copy
OUTPUT inserted.country_id, inserted.country_name, inserted.region_id
SELECT TOP(3) cid, cname, rid
FROM CountriesReg2
ORDER BY cid DESC;
```

Data Manipulation Language
INSERT

```
 1 DECLARE @CountriesTmp table(
 2     cid   char(2),
 3     cname varchar(30),
 4     rid   int
 5 );
 6
 7 INSERT countries_copy
 8     OUTPUT
 9         inserted.country_id,
10         inserted.country_name,
11         inserted.region_id
12     INTO @CountriesTmp
13 VALUES (N'SB', N'Serbia', 1),
14 (N'ME', N'Montenegro', 1);
15
16 SELECT * FROM @CountriesTmp;
17 GO
```

Data Manipulation Language
BULK INSERT

- BULK INSERT Imports a data file into a database table or view in a user-specified format in SQL Server

```
BULK INSERT
   { database_name.schema_name.table_or_view_name | schema_name.table_or_view_name | table_or_view_name }
      FROM 'data_file'
     [ WITH
    (
   [ [ , ] BATCHSIZE = batch_size ]
   [ [ , ] CHECK_CONSTRAINTS ]
   [ [ , ] CODEPAGE = { 'ACP' | 'OEM' | 'RAW' | 'code_page' } ]
   [ [ , ] DATAFILETYPE =
      { 'char' | 'native'| 'widechar' | 'widenative' } ]
   [ [ , ] DATA_SOURCE = 'data_source_name' ]
   [ [ , ] ERRORFILE = 'file_name' ]
   [ [ , ] ERRORFILE_DATA_SOURCE = 'data_source_name' ]
   [ [ , ] FIRSTROW = first_row ]
   [ [ , ] FIRE_TRIGGERS ]
   [ [ , ] FORMATFILE_DATA_SOURCE = 'data_source_name' ]
   [ [ , ] KEEPIDENTITY ]
   [ [ , ] KEEPNULLS ]
   [ [ , ] KILOBYTES_PER_BATCH = kilobytes_per_batch ]
   [ [ , ] LASTROW = last_row ]
   [ [ , ] MAXERRORS = max_errors ]
   [ [ , ] ORDER ( { column [ ASC | DESC ] } [ ,...n ] ) ]
   [ [ , ] ROWS_PER_BATCH = rows_per_batch ]
   [ [ , ] ROWTERMINATOR = 'row_terminator' ]
   [ [ , ] TABLOCK ]

   -- input file format options
   [ [ , ] FORMAT = 'CSV' ]
   [ [ , ] FIELDQUOTE = 'quote_characters']
   [ [ , ] FORMATFILE = 'format_file_path' ]
   [ [ , ] FIELDTERMINATOR = 'field_terminator' ]
   [ [ , ] ROWTERMINATOR = 'row_terminator' ]
    )]
```

Data Manipulation Language
UPDATE

- UPDATE Changes existing data in a table or view

```
-- Syntax for SQL Server and Azure SQL Database

[ WITH <common_table_expression> [...n] ]
UPDATE
    [ TOP ( expression ) [ PERCENT ] ]
    { { table_alias | <object> | rowset_function_limited
         [ WITH ( <Table_Hint_Limited> [ ...n ] ) ]
      }
      | @table_variable
    }
    SET
        { column_name = { expression | DEFAULT | NULL }
          | { udt_column_name.{ { property_name = expression
                                | field_name = expression }
                                | method_name ( argument [ ,...n ] )
                              }
          }
          | column_name { .WRITE ( expression , @Offset , @Length ) }
          | @variable = expression
          | @variable = column = expression
          | column_name { += | -= | *= | /= | %= | &= | ^= | |= } expression
          | @variable { += | -= | *= | /= | %= | &= | ^= | |= } expression
          | @variable = column { += | -= | *= | /= | %= | &= | ^= | |= } expression
        } [ ,...n ]

    [ <OUTPUT Clause> ]
    [ FROM{ <table_source> } [ ,...n ] ]
    [ WHERE { <search_condition>
            | { [ CURRENT OF
                  { { [ GLOBAL ] cursor_name }
                      | cursor_variable_name
                  }
                ]
              }
            }
    ]
    [ OPTION ( <query_hint> [ ,...n ] ) ]
[ ; ]

<object> ::=
{
    [ server_name . database_name . schema_name .
    | database_name .[ schema_name ] .
    | schema_name .
    ]
    table_or_view_name}
```



Data Manipulation Language
UPDATE

```
 1 DECLARE @EmpValues TABLE (
 2     SalaryBefore float,
 3     SalaryAfter float
 4 );
 5
 6 WITH BossTbl AS (
 7     SELECT DISTINCT manager_id id
 8     FROM employees WHERE manager_id IS NOT NULL
 9 )
10 UPDATE employees
11 SET salary = salary * 1.15
12     OUTPUT deleted.salary, inserted.salary
13     INTO @EmpValues
14 WHERE employee_id IN (SELECT id FROM BossTbl)
15
16 SELECT SalaryBefore, SalaryAfter
17 FROM @EmpValues
18 GO
```

Data Manipulation Language
DELETE

- DELETE removes one or more rows from a table or view in SQL Server

```
-- Syntax for SQL Server and Azure SQL Database

[ WITH <common_table_expression> [ ,...n ] ]
DELETE
    [ TOP ( expression ) [ PERCENT ] ]
    [ FROM ]
    { { table_alias
      | <object>
      | rowset_function_limited
      [ WITH ( table_hint_limited [ ...n ] ) ] }
      | @table_variable
    }
    [ <OUTPUT Clause> ]
    [ FROM table_source [ ,...n ] ]
    [ WHERE { <search_condition>
            | { [ CURRENT OF
                  { { [ GLOBAL ] cursor_name }
                      | cursor_variable_name
                  }
                ]
              }
            }
    ]
    [ OPTION ( <Query Hint> [ ,...n ] ) ]
[; ]

<object> ::=
{
    [ server_name.database_name.schema_name.
      | database_name. [ schema_name ] .
      | schema_name.
    ]
    table_or_view_name
}
```



Data Manipulation Language
DELETE

```
 1 DECLARE @CountriesTmp TABLE(
 2   cid varchar(2),
 3   cname varchar(20)
 4 );
 5
 6 WITH NoOffice AS (
 7   SELECT DISTINCT country_id AS cid FROM countries
 8   EXCEPT
 9   SELECT DISTINCT country_id AS cid FROM locations
10 )
11 DELETE FROM countries
12 OUTPUT deleted.country_id, deleted.country_name
13 INTO @CountriesTmp
14 WHERE country_id in (SELECT cid FROM NoOffice);
15
16 SELECT * FROM @CountriesTmp;
17 GO
```

Data Manipulation Language
SELECT

- SELECT retrieves rows from the database and enable

```
-- Syntax for SQL Server and Azure SQL Database

<SELECT statement> ::=
    [ WITH { [ XMLNAMESPACES ,] [ <common_table_expression> [,...n] ] } ]
    <query_expression>
    [ ORDER BY <order_by_expression> ]
    [ <FOR Clause>]
    [ OPTION ( <query_hint> [ ,...n ] ) ]
<query_expression> ::=
    { <query_specification> | ( <query_expression> ) }
    [  { UNION [ ALL ] | EXCEPT | INTERSECT }
        <query_specification> | ( <query_expression> ) [...n ] ]
<query_specification> ::=
SELECT [ ALL | DISTINCT ]
    [TOP ( expression ) [PERCENT] [ WITH TIES ] ]
    < select_list >
    [ INTO new_table ]
    [ FROM { <table_source> } [ ,...n ] ]
    [ WHERE <search_condition> ]
    [ <GROUP BY> ]
    [ HAVING < search_condition > ]
```

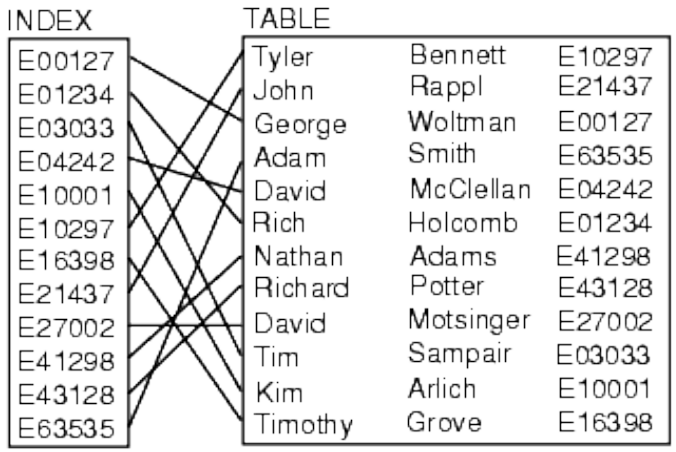


Data Manipulation Language
SELECT

```
1 SELECT TOP 5 first_name + ' ' + last_name
2 FROM EMPLOYEES
3 ORDER BY 1;
4 GO
```

Indexes

- A database index is a data structure that improves the speed of data retrieval operations on a database table at the cost of additional writes and storage space to maintain the index data structure
- Indexes are used to quickly locate data without having to search every row in a database table every time a database table is accessed
- Indexes can be created using one or more columns of a database table, providing the basis for both rapid random lookups and efficient access of ordered records





Indexes

Index type	Description
Hash	• Data is accessed through an in-memory hash table • Hash indexes consume a fixed amount of memory
Clustered	• Cluster Index sorts and stores the data rows of the table or view in order based on the clustered index key • The clustered index is implemented as a B-tree index structure that supports fast retrieval of the rows, based on their clustered index key values
Nonclustered	• A nonclusterd index can be defined on a table or view with a clustered index or on a heap (a table without a clustered index) • Each index row in the nonclustered index contains the nonclustered key value and a row locator • This locator points to the data row in the clustered index or heap having the key value. • The rows in the index are stored in the order of the index key values, but the data rows are not guaranteed to be in any particular order unless a clustered index is created on the table.
Memory-optimized Nonclustered	• For this index, the memory consumption is a function of the row count and the size of the index key columns

Indexes

Index type	Description
Unique	• A unique index ensures that the index key contains no duplicate values and therefore every row in the table or view is in some way unique. • Uniqueness can be a property of both clustered and nonclustered indexes.
Columnstore	• An in-memory columnstore index stores and manages data by using column-based data storage and column-based query processing • Columnstore indexes work well for data warehousing workloads that primarily perform bulk loads and read-only queries.
Index with included columns	• A nonclustered index that is extended to include non-key columns in addition to the key columns.
Index on computed columns	• An index on a column that is derived from the value of one or more other columns, or certain deterministic inputs.



Indexes

Index type	Description
Filtered	• An optimized nonclustered index, especially suited to cover queries that select from a well-defined subset of data. • It uses a filter predicate to index a portion of rows in the table. • A well-designed filtered index can improve query performance, reduce index maintenance costs, and reduce index storage costs compared with full-table indexes.
Spatial	• A spatial index provides the ability to perform certain operations more efficiently on spatial objects (spatial data) in a column of the geometry data type. • The spatial index reduces the number of objects on which relatively costly spatial operations need to be applied.
XML	• A shredded, and persisted, representation of the XML binary large objects (BLOBs) in the xml data type column.
Full-text	• A special type of token-based functional index that is built and maintained by the Microsoft Full-Text Engine for SQL Server • It provides efficient support for sophisticated word searches in character string data



Views

- A view
  - Is a virtual table whose contents are defined by a query
  - Consists of a set of named columns and rows of data (like a table)
  - Does not exist as a stored set of data values in a database (unless indexed)

- In a view, the rows and columns of data
  - Come from tables referenced in the query defining the view
  - Are produced dynamically when the view is referenced



Views

- Standard View
- Indexed View
  - A view that has been materialized, i.e., the view definition has been computed and the resulting data stored just like a table
- Partitioned view:
  - A partitioned view joins horizontally partitioned data from a set of member tables across one or more servers
  - This makes the data appear as if it is from one table
  - A view that joins member tables on the same instance of SQL Server is a local partitioned view
- System View views expose catalogue metadata



Views

```sql
-- standard view
CREATE VIEW ViewEmployeeName
AS
SELECT employee_id, first_name, last_name
FROM employees
GO

-- use the standard view
SELECT first_name, last_name FROM ViewEmployeeName
GO

--Set the options to support indexed views.
SET NUMERIC_ROUNDABORT OFF
SET ANSI_PADDING, ANSI_WARNINGS, CONCAT_NULL_YIELDS_NULL, ARITHABORT,
    QUOTED_IDENTIFIER, ANSI_NULLS ON
GO

--Create view with schemabinding.
CREATE VIEW ViewEmployeeName
WITH SCHEMABINDING
AS
SELECT employee_id, first_name, last_name
FROM [dbo].employees
GO

-- Create an index on the view to make the view indexed
CREATE UNIQUE CLUSTERED INDEX IDX_ViewEmployeeName ON ViewEmployeeName (employee_id)
GO

-- Use the indexed view
SELECT first_name, last_name FROM ViewEmployeeName
GO
```



Dynamic SQL

- Dynamic SQL
  - Not so fast, but can be anything
  - Usually used with Cursors
  - Syntax: EXEC @some_sql

EXEC ('SELECT * FROM employees')



Cursors in Transact-SQL

- A cursor is a data type for variables or stored procedure OUTPUT parameters that contain a reference to a cursor.
- The operations that can reference variables and parameters having a cursor data type are:
  - DECLARE, OPEN, FETCH, CLOSE, and DEALLOCATE cursor statements
  - Stored procedure output parameters
  - CURSOR_STATUS function



Cursors in Transact-SQL

```
DECLARE emp_cursor CURSOR FOR
  SELECT
    concat(e.first_name, ' ', e.last_name) AS nume_ang,
    d.department_name AS dept_ang
  FROM employees AS e
    LEFT JOIN  departments AS d ON e.department_id = d.department_id;
OPEN emp_cursor;
DECLARE @an varchar(50);
DECLARE @dn varchar(25);
FETCH NEXT FROM emp_cursor INTO @an, @dn;
WHILE (@@FETCH_STATUS <> -1)
BEGIN
  PRINT @an + ' works in ' + @dn;
  FETCH NEXT FROM emp_cursor INTO @an, @dn;
END;
PRINT 'Finished';
CLOSE emp_cursor;
DEALLOCATE emp_cursor;
GO
```

XQuery

- Transact-SQL supports a subset of the XQuery language
- XQuery is a language that can query structured or semi-structured XML data
- With the XML data type support provided in the Database Engine, documents can be stored in a database and then queried by using XQuery



XML DML

- The XML Data Modification Language is an extension of the XQuery language
- The XML DML provides a fully functional query and data-modification language that can be used for XML data
- It adds the following case-sensitive keywords to XQuery:
  - Insert
  - Delete
  - Replace value of



XML DML

```
 1 DROP TABLE IF EXISTS employees_xml;
 2
 3 CREATE TABLE employees_xml (xml_value XML);
 4
 5 DECLARE @XML_STRING NVARCHAR(MAX);
 6
 7 SET @XML_STRING = '<ROOT> ' + (
 8         SELECT e.name AS employee_name
 9             ,de.department_name AS employee_department
10             ,m.name AS manager_name
11             ,dm.department_name AS manager_department
12         FROM (SELECT employee_id, manager_id, department_id,
13                 concat(first_name, ' ', last_name) name
14               FROM employees) e
15         LEFT OUTER JOIN departments de
16             ON e.department_id = de.department_id
17         LEFT OUTER JOIN (SELECT employee_id, manager_id, department_id,
18                                 concat(first_name, ' ', last_name)
  name
19                          FROM employees) m
20             ON e.manager_id = m.employee_id
21         LEFT OUTER JOIN departments dm
22             ON m.department_id = dm.department_id
23         FOR XML AUTO
24         ) + '</ROOT>';
25
26 INSERT INTO employees_xml (xml_value) VALUES (@XML_STRING);
27 GO
```



XML DML

```
SET QUOTED_IDENTIFIER ON

DECLARE @doc XML, @iddoc INT

SELECT top(1) @doc = XML_VALUE FROM employees_xml

EXEC sp_xml_preparedocument @iddoc OUTPUT, @doc

SELECT
    e.x.value('@employee_name', 'varchar(max)')
        AS employee_name,
    de.x.value('@employee_department', 'varchar(max)')
        AS employee_department,
    m.x.value('@manager_name', 'varchar(max)')
        AS manager_name,
    dm.x.value('@manager_department', 'varchar(max)')
        AS manager_department
FROM
    @doc.nodes('/ROOT/e') AS e(x)
    OUTER APPLY e.x.nodes('de') AS de(x)
    OUTER APPLY de.x.nodes('m') AS m(x)
    OUTER APPLY m.x.nodes('dm') AS dm(x);
GO
```



Graph Query

- Using a graph, data can be represented by
  - Relationships (edges)
  - Entities (nodes)

- Graph structures
  - Make finding and expressing relationships easier
  - Help to generate better insights from interconnected data

Graph Query

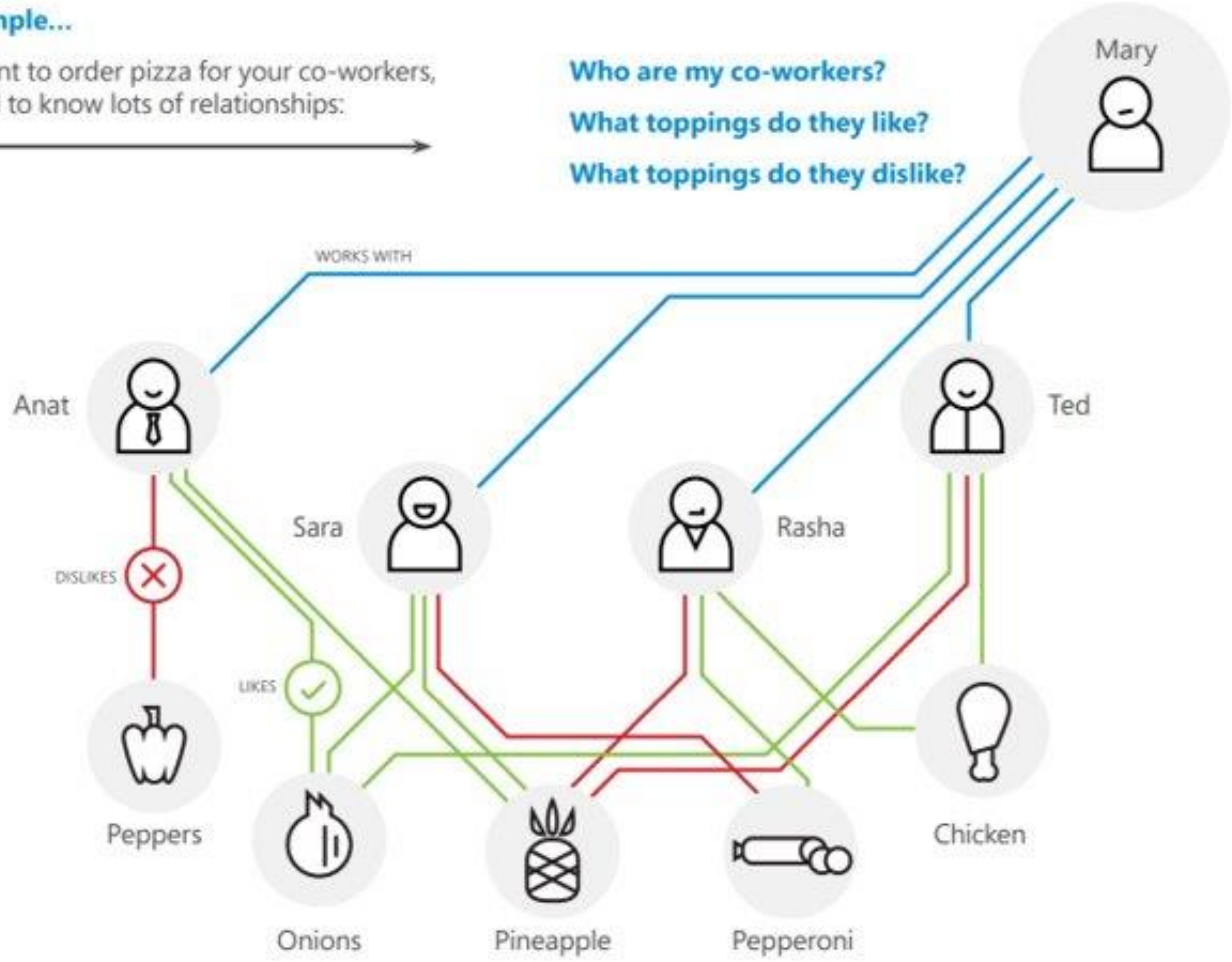
For example...

If you want to order pizza for your co-workers, you need to know lots of relationships:

Who are my co-workers?

What toppings do they like?

What toppings do they dislike?





Graph Query

TRADITIONAL SQL QUERY: VS **GRAPH QUERY:**

```
SELECT DISTINCT Toppings.topping
FROM dbo.toppings AS Toppings
WHERE Toppings.id IN
      (SELECT Likes.toppingID
      FROM dbo.relToppingLikes AS Likes
      WHERE Likes.peopleID IN
            (SELECT Coworkers.coworkerID
            FROM dbo.relCoworkers AS Coworkers
            WHERE Coworkers.personID IN
                  (SELECT Person.id
                  FROM dbo.people AS Person
                  WHERE Person.firstName = 'Mary')))
      AND Toppings.id NOT IN
      (SELECT DisLikes.toppingID
      FROM dbo.relToppingDisLikes AS DisLikes
      WHERE DisLikes.peopleID IN
            (SELECT Coworkers.coworkerID
            FROM dbo.relCoworkers AS Coworkers
            WHERE Coworkers.personID IN
                  (SELECT Person.id
                  FROM dbo.people AS Person
                  WHERE Person.firstName = 'Mary')))
```

```
SELECT DISTINCT Topping.topping
FROM gPeople AS Person, gPeople AS Coworker,
eWorkswith, eLikes, eDisLikes, gToppings AS Topping
WHERE MATCH(Person-(eWorkswith)->Coworker-(eLikes)-
>Topping)
AND Topping.ID NOT IN
      (SELECT DISTINCT Topping.ID
      FROM gPeople AS Coworker, eWorkswith,
      eDisLikes, gToppings AS Topping
      WHERE MATCH(Person-(eWorkswith)->Coworker-
      (eDisLikes)->Topping))
AND Person.firstName='Mary'
```



User-Defined Functions

- SQL Server supports user-defined functions (UDFs)
- UDFs are routines that
  - Accept parameters
  - Perform an action, such as a complex calculation
  - Return the result of that action as a value
- The return value can either be a single scalar value or a result set.
- Function features:
  - READ-ONLY
  - Faster execution
  - Can reduce network traffic



User-Defined Functions

```sql
CREATE OR ALTER FUNCTION ufnIsBoss(@eid int)
RETURNS varchar(3)
AS
BEGIN
  DECLARE @ok int;
  DECLARE @isBoss varchar(3) = 'Yes';
  SELECT @ok = count(*)
  FROM employees
  WHERE manager_id = @eid;
  IF (@ok = 0)
    SET @isBoss = 'No';
  RETURN @isBoss;
END;
GO

SELECT
    first_name,
    last_name,
    [dbo].ufnIsBoss(employee_id)
FROM employees;
GO
```



User-Defined Functions

```sql
CREATE FUNCTION ufnGetSubordinate(@eid int)
RETURNS TABLE
AS
RETURN
(
    SELECT concat(first_name, ' ', last_name) AS subordinate
    FROM employees
    WHERE manager_id = @eid;
);
GO

SELECT * FROM ufnGetSubordinate(100)
GO
```



Stored Procedures

- A stored procedure in SQL Server is
  - A group of one or more Transact-SQL statements
  - A reference to a Microsoft .NET Framework common runtime language (CLR) method
- Procedures:
  - Accept input parameters and return multiple values
  - Contain programming statements that perform operations in the database
  - Return a status value to a calling program to indicate success or failure (and the reason for failure)
- Features:
  - Reduced server/client network traffic
  - Stronger security
  - Reuse of code
  - Improved performance



Stored Procedures

```
CREATE OR ALTER PROCEDURE usp_divide(
  @a decimal,
  @b decimal,
  @c decimal OUT
)
AS
BEGIN
  BEGIN TRY
    SET @c = @a / @b
  END TRY
  BEGIN CATCH
    PRINT 'Error Number    ' + STR(ERROR_NUMBER())
    PRINT 'Error Severity  ' + STR(ERROR_SEVERITY())
    PRINT 'Error State     ' + STR(ERROR_STATE())
    PRINT 'Error Procedure ' + ERROR_PROCEDURE()
    PRINT 'Error Line      ' + STR(ERROR_LINE())
    PRINT 'Error Message   ' + ERROR_MESSAGE()
  END CATCH
END
GO

DECLARE @r decimal;
EXEC usp_divide 10, 2, @r output;
PRINT @r;
GO

DECLARE @r decimal;
EXEC usp_divide 10, 0, @r output;
PRINT @r;
GO
```



Triggers

- Event-driven types of instructions that are used to ensure the integrity of the database
- DDL Triggers fire in response to a DDL event
- DML Triggers
  - Is a special type of stored procedure
  - Automatically takes effect when a DML event that affects the table or view defined in the trigger takes place (i.e., INSERT, UPDATE, or DELETE) => AFTER triggers
- DML triggers can be used to
  - Enforce business rules
  - Data integrity
  - Query other tables
  - Include complex Transact-SQL statements



DML Triggers

```
CREATE OR ALTER TRIGGER UpdateSalaryOrCommision
ON employees
FOR UPDATE
AS
   DECLARE @sal float
   DECLARE @comm float
   DECLARE @emp varchar(50)
   DECLARE @message varchar(255)
   SELECT
      @emp = ins.first_name + ' ' + ins.last_name,
      @sal = ins.salary,
      @comm = ins.commission_pct
   FROM INSERTED ins

   IF UPDATE(salary) AND UPDATE(commission_pct)
      SET @message = concat('Updated ', @emp, ' salary to ', @sal, ' and comm to ',
@comm)
   ELSE
   BEGIN
      IF UPDATE(salary)
         SET @message = concat('Updated ', @emp, ' salary to ', @sal)
      ELSE
         SET @message = concat('Updated ', @emp, ' comm to ', @comm)
   END
   PRINT @message
GO

UPDATE employees SET salary = 5000 WHERE employee_id = 100;
GO

UPDATE employees SET commission_pct = 0.5 WHERE employee_id = 100;
GO

UPDATE employees SET salary = 5000, commission_pct = 0.5 WHERE employee_id = 100;
GO
```

References

- Microsoft SQL Server 2019 Technical Documentation
https://learn.microsoft.com/en-us/sql/sql-server/?view=sql-server-ver15

- Microsoft SQL Server 2022 Technical Documentation
https://learn.microsoft.com/en-us/sql/sql-server/?view=sql-server-ver16